

 AI AGENTS

10 Components Every Production AI Agent Must Have

An LLM is not an agent.
Here's the architecture that makes it **actually work in
production.**



AI Coach John

FOLLOW



 Repost

LLM vs Agent - What's the Difference?

While many use the terms interchangeably, an LLM is merely the engine of an AI Agent. To build reliable systems that solve real business problems, you must wrap the LLM in a robust architecture that handles state, tools, and execution logic.

✗ AN LLM ALONE

Reads input → generates text

- Stateless (forgets after call)
- No tools (can't act)
- No planning (one-shot)
- No recovery from errors
- No memory across sessions
- No autonomy

✓ AN AGENT SYSTEM

LLM + architecture around it

- Memory layer (stateful)
- Tool registry (can act)
- Planner (multi-step)
- Error recovery
- Cross-session continuity
- Can be autonomous

THE 10 COMPONENTS

- | | |
|--------------------------|-----------------------------|
| 01 Planner | 06 Reflection Loop |
| 02 Tool Registry | 07 Evaluator |
| 03 Memory Layer | 08 Human-in-the-Loop |
| 04 Tool Selection | 09 Error Recovery |
| 05 Tool Execution | 10 Observability |



Planner - Break Goal Into Steps

❗ WHAT IT DOES

Takes the user's goal and decomposes it into a sequence of actionable steps. The LLM is the planner - prompted to think before acting.

Without a planner: Agent takes random actions and gets stuck in loops.

⚡ 3 PLANNING STRATEGIES

- **ReAct:** Reason then Act, step by step
- **ToT:** Tree of Thoughts - explore branches
- **Chain-of-Plan:** Full plan before execution

</> REACT PLANNER (ANTHROPIC SDK)

```
from anthropic import Anthropic
client = Anthropic()

def plan_task(goal: str) → list[str]:
    response =
    client.messages.create(
        model="claude-opus-4-8",
        max_tokens=1000,
        system="""You are a task
planner.
        Break goal into steps
(JSON).""",
        messages=[{"role": "user",
"content": goal}]
    )
    return
    json.loads(response.content[0].text)
    ["steps"]
    # Plan: expensive model. Execute:
    cheap model.
```

★ **Design rule:** Use your most capable model for planning. Use a faster model for execution. One expensive planning call beats 10 confused execution calls.



AI Coach John

FOLLOW



🔄 Repost

Tool Registry - The Agent's Hands

Every action the agent can take is a tool. The registry is the catalog of everything it's allowed to do.

</> TOOL DEFINITION + REGISTRATION (ANTHROPIC SDK)

```
tools = [{
  "name": "search_kb",
  "description": "Search internal KB",
  "input_schema": {
    "type": "object",
    "properties": { "query": { "type": "string" } },
    "required": ["query"]
  }
}, {
  "name": "execute_sql",
  "description": "Run safe SQL on DB",
  "input_schema": {
    "type": "object",
    "properties": { "sql": { "type": "string" } },
    "required": ["sql"]
  }
}]
# Pass tools list to API call
response = client.messages.create(
  model="claude-haiku", tools=tools, messages=[ ... ]
)
```

 Web / KB search SQL / vector query Code interpreter Email / calendar API calls File read/write

AI Coach John

FOLLOW



 Repost

Memory Layer - State Across Steps

3 TYPES YOU NEED

- 1. Working memory:** Current step results, scratchpad for reasoning
- 2. Session memory:** Full conversation history this session (Redis)
- 3. Long-term memory:** Facts, preferences from past sessions (Vector DB)

⚠️ WHAT BREAKS

No memory → agent forgets what it found in step 2 by step 5.

No pruning → context window fills up → agent crashes at step 8 of a 10-step task.

</> AGENT MEMORY MANAGER

```
class AgentMemory:
    def __init__(self,
        session_id: str):
        self.session = session_id
        self.working = {}
        self.results = []

    def save_result(self, step:
        str, res: str):
        self.results.append({
            "step": step, "res":
            res[:500]
        })
        self.working["last"] = res

    def get_context(self) →
        str:
        # Return only the last 5
        steps
        return "\n".join([
            f"{s['step']}:
            {s['res']}"
            for s in
            self.results[-5:]
        ])
```



Tool Selection & Execution

04 - TOOL SELECTION

LLM chooses which tool to call based on the current step. Claude returns `tool_use` blocks natively.

05 - EXECUTION

Your code routes the LLM's decision to the actual tool function and returns the result. Never let the LLM execute code directly.

</> TOOL SELECTION + EXECUTION LOOP

```
def execute_tools(response, tool_map):
    results = []
    for block in response.content:
        if block.type == "tool_use":
            fn = tool_map.get(block.name)
            try:
                out = fn(**block.input)
                results.append({
                    "type": "tool_result",
                    "tool_use_id": block.id, "content": str(out)
                })
            except Exception as e:
                results.append({
                    "type": "tool_result",
                    "tool_use_id": block.id,
                    "content": str(e), "is_error": True
                })
    return results

TOOL_MAP = {"search": kb_search, "sql": safe_sql}
```



Reflection Loop - Self-Correction Before Output



</> REFLECTION AGENT PATTERN

```

def reflect_on_step(task: str, result: str) → dict:
    eval_resp = client.messages.create(
        model="claude-haiku", max_tokens=300,
        system="""Evaluate if result completes task.
        Return JSON: {"verdict": "PASS|RETRY|ESCALATE",
        "reason": " ... ", "feedback": "what to fix"}""",
        messages=[{"role": "user",
                    "content": f"Task: {task}\nResult: {result}"}]
    )
    return json.loads(eval_resp.content[0].text)
  
```

✓ PASS

Continue to next step or output result

🔄 RETRY

Re-run step with reflection feedback injected



AI Coach John

FOLLOW



🔄 Repost

Evaluator & Human-in-the-Loop

07 - EVALUATOR

Scores the final output against a rubric before delivery. Catches bad answers **before** the user sees them.

Metrics: Relevance, Groundedness, Completeness, Safety

</> RAGAS AUTO-EVAL

```
from ragas import evaluate
from ragas.metrics import (
    faithfulness,
    answer_relevancy)

scores = evaluate(
    dataset=Dataset.from_dict({
        "question": [q],
        "answer": [ans],
        "contexts": [ctx]
    }),
    metrics=[faithfulness,
             answer_relevancy]
)
# Block if faithfulness < 0.8
```

08 - HUMAN-IN-THE-LOOP

Critical actions require human approval before execution.

Gate these:

- > Send email to client
- > Deploy to production
- > Delete records
- > Spend money
- > Confidence < 80%

</> APPROVAL GATE

```
RISKY = ["send_email",
         "deploy", "delete"]

def needs_approval(tool):
    return tool in RISKY

if needs_approval(tool_name):
    approved = await
    request_human(
        f"Approve: {tool_name}?"
    )
    if not approved:
        return "Action cancelled"
```



Error Recovery - What Happens When It Fails?

Agents will fail. The question is: does the failure kill the task, or does the agent recover and finish?

</> RESILIENT ERROR HANDLER

```
class AgentErrorHandler:
    def __init__(self, max_retries: int = 3): self.max_retries = max_retries

    def handle(self, err: Exception, step: str, attempt: int) → dict:
        if attempt < self.max_retries:
            wait = 2 ** attempt # Exponential backoff
            time.sleep(wait)
            return {"action": "retry", "msg": f"Failed '{step}': {err}. Retry
in {wait}s"}
        return {"action": "fallback", "msg": f"Skipping '{step}': Max retries
hit", "partial": "Unable to complete"}
```

Retry

With backoff

Fallback

Use alt tool

Escalate

Human review



Non-negotiable: Set max_iterations on every agent. Without it, a stuck agent costs you \$50 in API calls.



AI Coach John

FOLLOW



 Repost

Observability - See Inside the Black Box

📌 WHAT TO TRACE

- Every LLM call (in/out/tokens)
- Every tool call + result
- Step durations (find bottlenecks)
- Retry count per step
- Total cost per run
- Human approval events

🔧 TOOLS

LangSmith - best for LangChain agents

Langfuse - open source, self-hostable

Helicone - proxy-based, any LLM

Arize Phoenix - ML + LLM observability

</> LANGSMITH TRACING

```
from langsmith import traceable
import os

os.environ["LANGCHAIN_TRACING_V2"]
= "true"
os.environ["LANGCHAIN_API_KEY"] =
LSKEY

@traceable(
    run_type="chain",
    name="AgentStep"
)
def agent_step(
    step: str,
    context: dict
) → str:
    # This call auto-traced
    result = llm.invoke(step)
    return result

# Every call now appears in
# LangSmith dashboard with:
# - Inputs/Outputs
# - Latency breakdown
# - Token usage + cost
# - Error traces
```



How One Request Flows Through All 10 Components

- 1 **Memory Layer** loads context: user profile + relevant past sessions
- 2 **Planner** breaks the user goal into 4 steps with success criteria
- 3 **Tool Selection** picks the right tool for step 1 from Tool Registry
- 4 **Tool Execution** routes to actual function, returns result
- 5 **Reflection Loop** checks: did this step succeed? If not, retry with feedback
- 6 **Steps 2–4 repeat** per plan step. Memory updated each time
- 7 **Evaluator** scores final output → **HITL gate** if risky action → **Observability** logs everything



❌ ANTI-PATTERNS

Mistakes That Break Production Agents

❌ No iteration limit

Agent loops indefinitely → \$50 API bill. Always set `max_iterations=10`.

❌ No memory compression

Context fills at step 8 of a 10-step task. Agent crashes right before the finish line.

❌ Tools with side effects, no approval gate

Agent sends emails, deletes records, deploys code - without any human review. Human-in-the-Loop is not optional for these actions.

❌ No observability

Agent fails in production. You have no idea which step, what the input was, or why. Add tracing from day one, not after the first production incident.

❌ No error recovery strategy

Tool call fails → agent halts completely. Define retry, fallback, and escalate behaviors for every tool.



AI Coach John

FOLLOW



↺ ↻ Repost

The Full Production **Agent Stack**

COMPONENT	RECOMMENDED	ALTERNATIVE
LLM (Plan)	Claude Opus 4.8	GPT-4o
LLM (Execute)	Claude Haiku 4.5	GPT-4o-mini
Orchestration	LangGraph	CrewAI, AutoGen
Session Memory	Redis	DynamoDB
Long-term Memory	Pinecone	pgvector
Tool Framework	Anthropic tool_use	LangChain Tools
Observability	LangSmith	Langfuse
Evaluation	RAGAS	UpTrain



Build these 10 components. You don't have an AI assistant - you have an AI engineer.



AI Coach John

FOLLOW



PROTBRIDGE
Drive For Better Future

 **Repost**

AI Agent Architecture Cheatsheet

#	COMPONENT	WHAT IT SOLVES	PRIORITY
01	Planner	Random aimless actions	P0
02	Tool Registry	Agent can't act on world	P0
03	Memory Layer	Forgets context mid-task	P0
04	Tool Selection	Uses wrong tool for job	P0
05	Tool Execution	LLM can't actually call APIs	P0
06	Reflection Loop	Errors propagate undetected	P1
07	Evaluator	Bad answers reach users	P1
08	HITL	Agent takes dangerous actions	P1
09	Error Recovery	One failure kills entire task	P2
10	Observability	Blind to production failures	P1

 **P0 components (01–05) must exist before you call it a production agent. No exceptions.**

